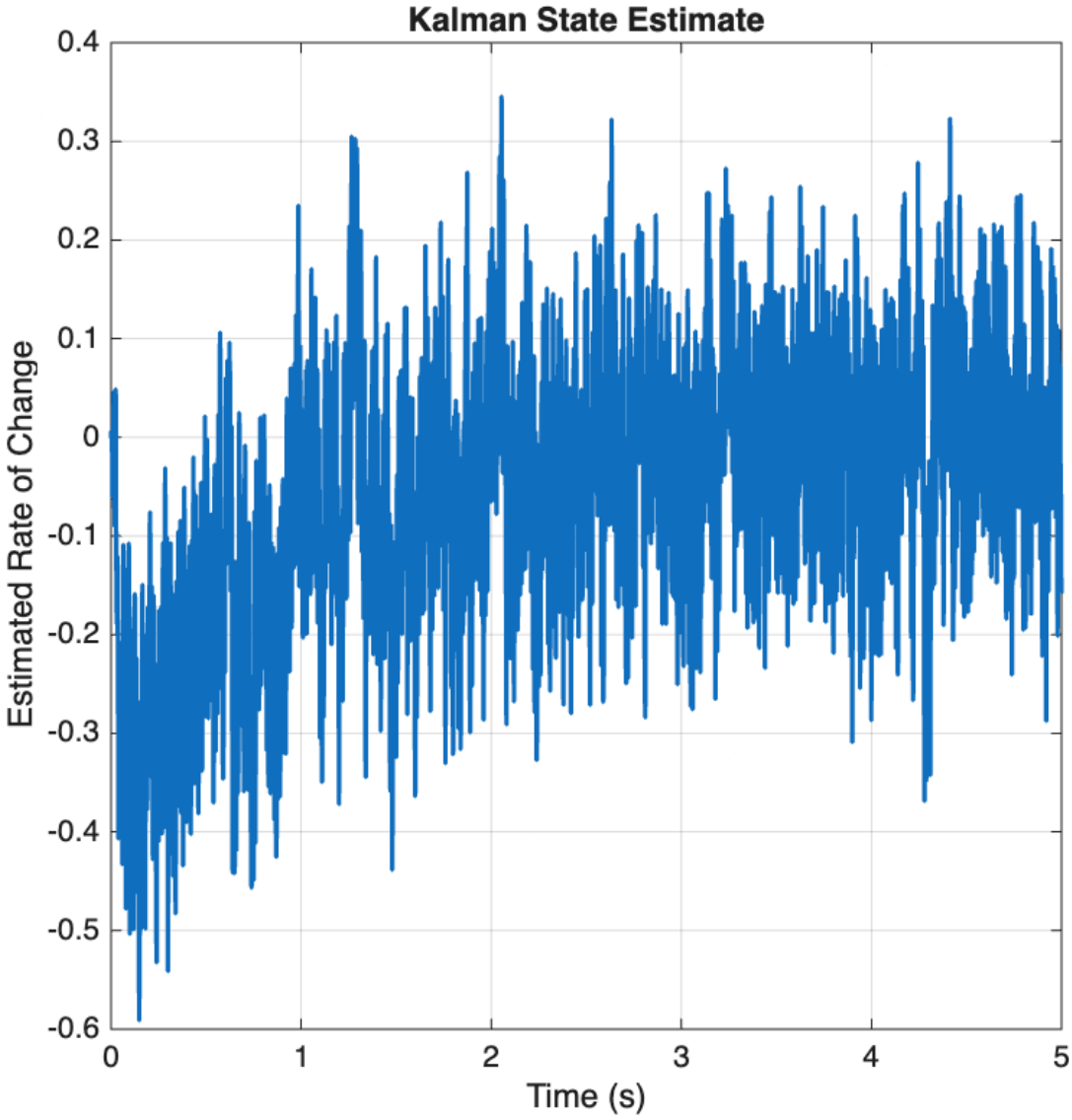
```
clc;
clear;
close all;
%% ============================================================
% KALMAN FILTER BASED SIGNAL DENOISING
% Digital Signal Processing Project
% ============================================================
%% 1. SIGNAL PARAMETERS
Fs = 1000;                % Sampling frequency in Hz
T = 1/Fs;                 % Sampling period
L = 5000;                 % Number of samples
t = (0:L-1)*T;
%% 2. GENERATE ORIGINAL SIGNAL
f1 = 100;                 % First frequency
f2 = 250;                 % Second frequency
f3 = 400;                 % Third frequency
x = sin(2*pi*f1*t) ...
  + 0.5*sin(2*pi*f2*t) ...
  + 0.3*sin(2*pi*f3*t);
%% 3. ADD GAUSSIAN NOISE
SNR_input = 5;            % Input SNR in dB
% Calculate signal power
signal_power = mean(x.^2);
% Calculate required noise power
noise_power = signal_power / (10^(SNR_input/10));
% Generate Gaussian noise
noise = sqrt(noise_power)*randn(size(x));
% Noisy signal
y = x + noise;
%% ============================================================
% 4. KALMAN FILTER INITIALIZATION
% ============================================================
% State vector:
%
% X = [Signal amplitude
%      Rate of change]
F = [1 T;
   0 1];
% Measurement matrix
H = [1 0];
% Process noise covariance
Q = [0.001 0;
   0     0.001];
% Measurement noise covariance
```

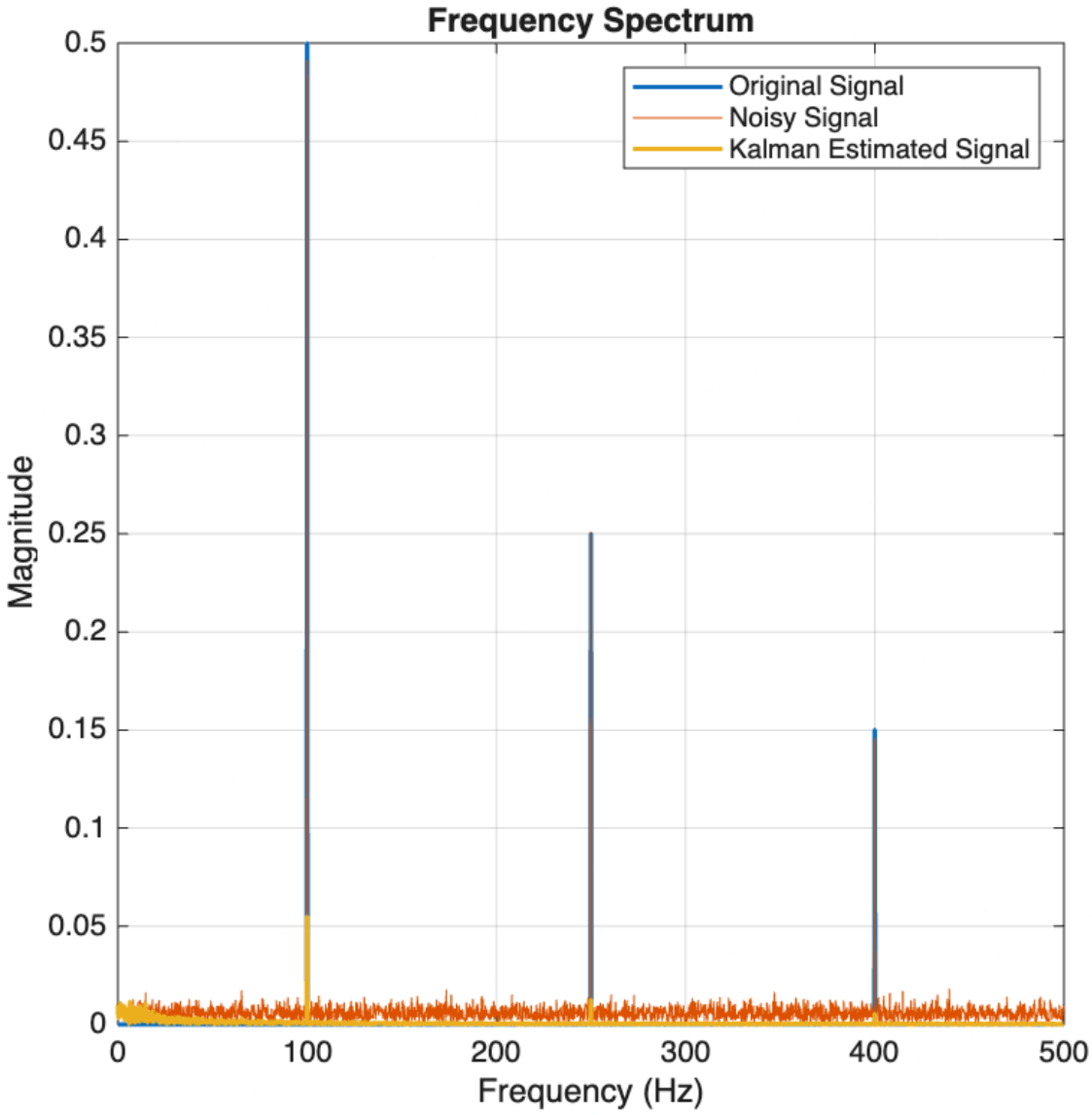
```
R = noise_power;
% Initial error covariance
P = eye(2);
% Initial state
X = [y(1);
    0];
% Storage for Kalman output
x_kalman = zeros(1,L);
% Storage for estimated velocity
velocity_estimate = zeros(1,L);
%% ============================================================
% 5. KALMAN FILTER
% ============================================================
for k = 1:L
   % -------------------------
   % Prediction
   % -------------------------
   X_pred = F * X;
   P_pred = F * P * F' + Q;
   % -------------------------
   % Measurement
   % -------------------------
   Z = y(k);
   % -------------------------
   % Kalman Gain
   % -------------------------
   K = P_pred * H' / ...
      (H * P_pred * H' + R);
   % -------------------------
   % Correction
   % -------------------------
   X = X_pred + K * (Z - H * X_pred);
   P = (eye(2) - K * H) * P_pred;
   % -------------------------
   % Store results
   % -------------------------
   x_kalman(k) = X(1);
   velocity_estimate(k) = X(2);
end
%% ============================================================
% 6. MSE CALCULATION
% ============================================================
mse_noisy = mean((x - y).^2);
mse_kalman = mean((x - x_kalman).^2);
```

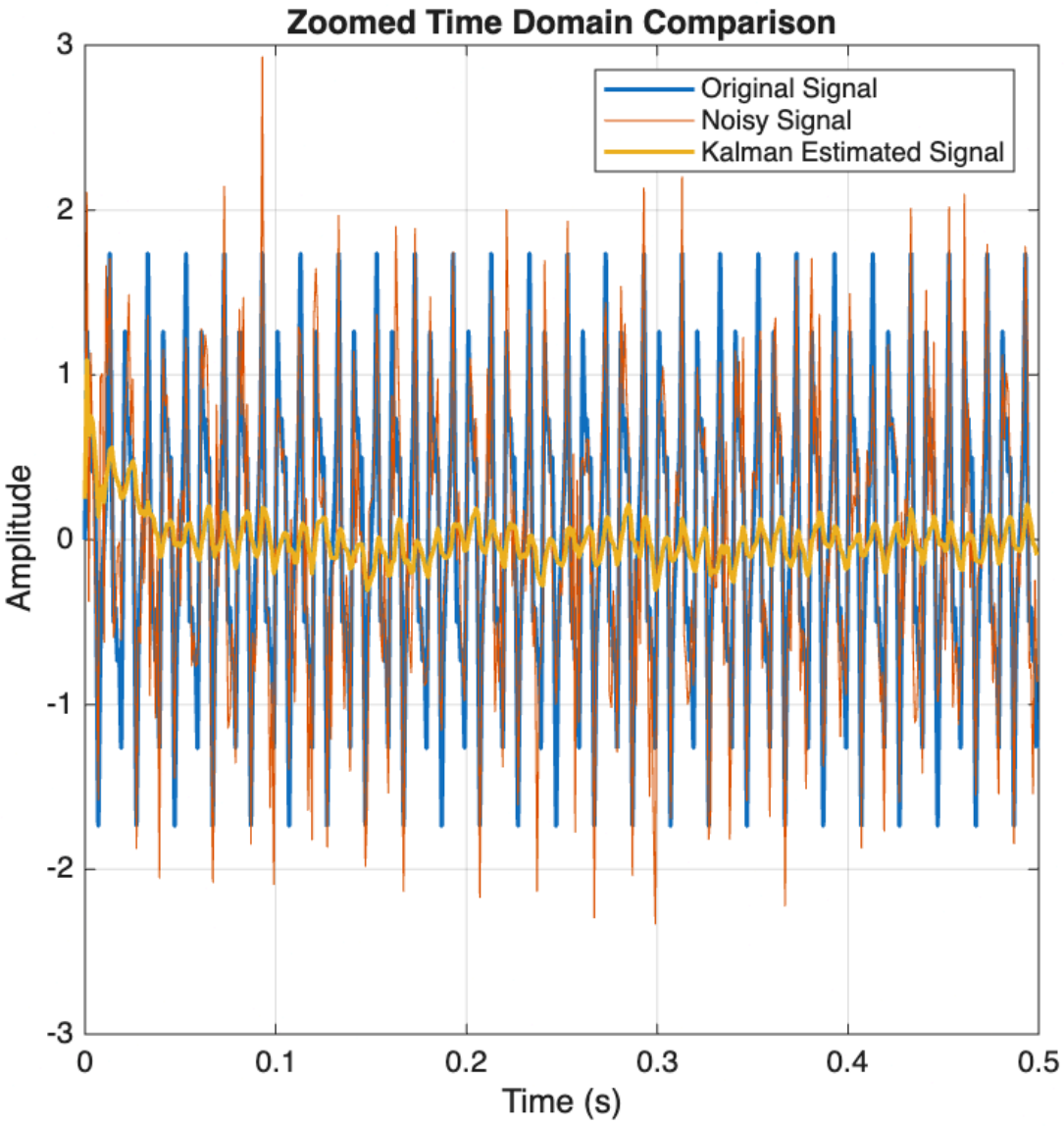
```
%% ============================================================
% 7. RMSE CALCULATION
% ============================================================
rmse_noisy = sqrt(mse_noisy);
rmse_kalman = sqrt(mse_kalman);
%% ============================================================
% 8. SNR CALCULATION
% ============================================================
error_noisy = x - y;
error_kalman = x - x_kalman;
power_error_noisy = mean(error_noisy.^2);
power_error_kalman = mean(error_kalman.^2);
snr_noisy = 10*log10(signal_power/power_error_noisy);
snr_kalman = 10*log10(signal_power/power_error_kalman);
snr_improvement = snr_kalman - snr_noisy;
%% ============================================================
% 9. DISPLAY NUMERICAL RESULTS
% ============================================================
fprintf('\n');
fprintf('================================================\n');
fprintf('       KALMAN FILTER DSP PROJECT\n');
fprintf('================================================\n');
fprintf('\nSignal Parameters\n');
fprintf('Sampling Frequency = %.0f Hz\n',Fs);
fprintf('Number of Samples  = %d\n',L);
fprintf('Input SNR          = %.2f dB\n',SNR_input);
fprintf('\n------------------------------------------------\n');
fprintf('MSE RESULTS\n');
fprintf('------------------------------------------------\n');
fprintf('Noisy Signal MSE   = %.6f\n',mse_noisy);
fprintf('Kalman Signal MSE  = %.6f\n',mse_kalman);
fprintf('\n------------------------------------------------\n');
fprintf('RMSE RESULTS\n');
fprintf('------------------------------------------------\n');
fprintf('Noisy Signal RMSE  = %.6f\n',rmse_noisy);
fprintf('Kalman Signal RMSE = %.6f\n',rmse_kalman);
fprintf('\n------------------------------------------------\n');
fprintf('SNR RESULTS\n');
fprintf('------------------------------------------------\n');
fprintf('Noisy Signal SNR   = %.2f dB\n',snr_noisy);
fprintf('Kalman Output SNR  = %.2f dB\n',snr_kalman);
fprintf('SNR Improvement    = %.2f dB\n',snr_improvement);
fprintf('\n================================================\n');
%% ============================================================
```

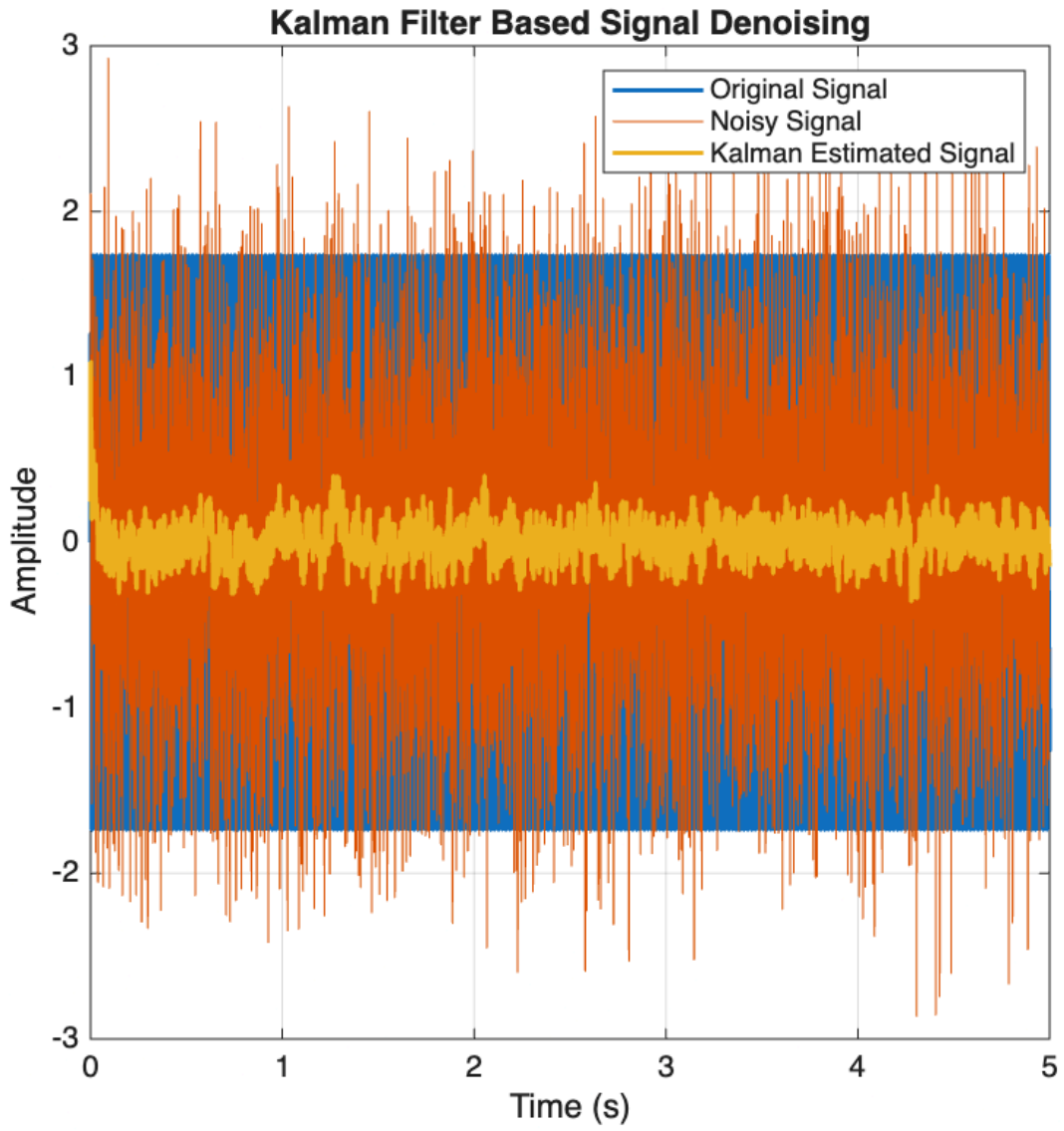
```
% 10. TIME DOMAIN PLOT
% ========================================================
figure;
plot(t,x,'LineWidth',1.5);
hold on;
plot(t,y);
plot(t,x_kalman,'LineWidth',1.5);
grid on;
xlabel('Time (s)');
ylabel('Amplitude');
title('Kalman Filter Based Signal Denoising');
legend('Original Signal', ...
    'Noisy Signal', ...
    'Kalman Estimated Signal');
%% ========================================================
% 11. ZOOMED TIME DOMAIN PLOT
% ========================================================
N_zoom = 500;
figure;
plot(t(1:N_zoom),x(1:N_zoom),'LineWidth',1.5);
hold on;
plot(t(1:N_zoom),y(1:N_zoom));
plot(t(1:N_zoom),x_kalman(1:N_zoom),'LineWidth',1.5);
grid on;
xlabel('Time (s)');
ylabel('Amplitude');
title('Zoomed Time Domain Comparison');
legend('Original Signal', ...
    'Noisy Signal', ...
    'Kalman Estimated Signal');
%% ========================================================
% 12. FFT ANALYSIS
% ========================================================
X_fft = fft(x);
Y_fft = fft(y);
K_fft = fft(x_kalman);
% Frequency axis
f = (0:L-1)*(Fs/L);
% Magnitude
X_mag = abs(X_fft)/L;
Y_mag = abs(Y_fft)/L;
K_mag = abs(K_fft)/L;
%% ========================================================
% 13. FREQUENCY DOMAIN PLOT
```

```
% ============================================================
figure;
plot(f(1:L/2),X_mag(1:L/2),'LineWidth',1.5);
hold on;
plot(f(1:L/2),Y_mag(1:L/2));
plot(f(1:L/2),K_mag(1:L/2),'LineWidth',1.5);
grid on;
xlabel('Frequency (Hz)');
ylabel('Magnitude');
title('Frequency Spectrum');
legend('Original Signal', ...
    'Noisy Signal', ...
    'Kalman Estimated Signal');
%% ============================================================
% 14. VELOCITY ESTIMATE
% ============================================================
figure;
plot(t,velocity_estimate,'LineWidth',1.5);
grid on;
xlabel('Time (s)');
ylabel('Estimated Rate of Change');
title('Kalman State Estimate');
%% ============================================================
% END
% ============================================================
print(1, 'kalman_time_domain.eps', '-depsc', '-r300');
print(2, 'kalman_zoomed.eps', '-depsc', '-r300');
print(3, 'kalman_frequency_spectrum.eps', '-depsc', '-r300');
print(4, 'kalman_state_estimate.eps', '-depsc', '-r300');
```


Kalman State Estimate
Estimated Rate of Change
Time (s)
0.4
0.3
0.2
0.1
0
-0.1
-0.2
-0.3
-0.4
-0.5
-0.6
0
1
2
3
4
5


Frequency Spectrum
Original Signal
Noisy Signal
Kalman Estimated Signal
Magnitude
Frequency (Hz)
0
0.05
0.1
0.15
0.2
0.25
0.3
0.35
0.4
0.45
0.5
0
100
200
300
400
500


Zoomed Time Domain Comparison
Original Signal
Noisy Signal
Kalman Estimated Signal
Amplitude
Time (s)
3
2
1
0
-1
-2
-3
0
0.1
0.2
0.3
0.4
0.5


Kalman Filter Based Signal Denoising
Original Signal
Noisy Signal
Kalman Estimated Signal
Amplitude
Time (s)
3
2
1
0
-1
-2
-3
0
1
2
3
4
5